`ifunnel`: A Four-Stage Pipeline for CVaR-Targeted Portfolio Construction, Measured Stage by Stage

Thomas Schmelzer*

August 31, 2026

Abstract

`ifunnel` builds portfolios by pushing a large fund universe through four stages: select a diversified subset by minimum spanning tree or by clustering, estimate moments and generate scenarios, derive a risk target from a benchmark, and solve a convex programme that maximises expected return subject to a Conditional Value-at-Risk limit. This paper measures each stage separately on the 754-fund Danish universe the repository ships, selecting on 2017–2020 and testing on 2021–2024. The largest effect we can measure anywhere in the pipeline is not a modelling choice at all: extending the selection window to overlap the evaluation period lifts the clustering selector's equal-weight Sharpe ratio from +0.17 to +1.02, because that selector ranks funds by realised Sharpe ratio, and nothing in its signature or documentation says the window must end first. The second largest is the optimisation stage, which on this window costs between 1.0 and 1.5 Sharpe against equal weight on the same names, in all six model and scenario combinations we ran; two mechanisms account for it, an objective that extrapolates the training-window mean and a risk budget taken from the benchmark rather than from the subset — the benchmark's 4-week tail is 1.29 to 2.00 times the subset's, and the optimised portfolios duly come back at two to three times the volatility of equal weight. Three findings are defects rather than surprises. The Monte Carlo scenario branch produces essentially Gaussian 4-week returns (skewness +0.10, excess kurtosis +0.10) where the bootstrap branch keeps most of the historical tail (−1.05, +3.17), understating the CVaR of the same portfolio by 33%; because the target is *always* bootstrapped, that mismatch enlarges the risk budget by half rather than cancelling. The Portfolio Diversification Index is computed from the wrong spectrum and misses its last term, returning $n - 1$ instead of n for n independent assets and NaN for a perfectly correlated block, and on real data it reverses the advice: the coded index says diversification improves through five MST passes, the correct one says it peaks after two. And the CVaR path is unseeded, so an identical call returns Sharpe ratios spanning up to 0.76 — wider than the differences between the configurations it is used to compare.

1 Introduction

The portfolio problem that Markowitz (1952) posed has a well-known practical failure mode: the optimiser is a magnifying glass held over the estimation error in its inputs (Michaud, 1989; Best and Grauer, 1991), and the remedy is usually taken to be either better estimates (Ledoit and Wolf, 2004; Jorion, 1986) or fewer decisions (DeMiguel et al., 2009). Replacing variance with a coherent tail measure (Artzner et al., 1999) — Conditional Value-at-Risk, which Rockafellar and Uryasev (2000) and Krokhmal et al. (2002) showed can be optimised as a linear programme — changes what is being controlled but not that fact.

`ifunnel` is an implementation of a complete pipeline built on that machinery, aimed at a universe too large to optimise over directly: 754 Danish mutual funds and ETFs, weekly, from 2013. Its four stages are

*`https://github.com/tschm/funnel`

1. **Selection.** Reduce the universe to a few dozen names, either by repeatedly taking the leaves of a minimum spanning tree over the correlation graph (Mantegna, 1999; Onnela et al., 2003), or by hierarchical clustering followed by picking the best-performing names in each cluster.

2. **Scenarios.** Estimate rolling means and Ledoit–Wolf-shrunk covariances, and generate 4-week return scenarios either by Monte Carlo from those moments or by bootstrapping historical weeks (Efron, 1979).

3. **Targets.** Compute, for each rebalancing period, the CVaR of an equal-weight portfolio of the chosen *benchmark*, and use it as the risk limit.

4. **Optimisation.** Maximise scenario expected return subject to that CVaR limit, a budget constraint, linear transaction costs and a concentration cap, rebalancing every four weeks.

Each stage is individually defensible and each is a published technique. The question this paper asks is which of them the result actually depends on. That question is worth asking of any pipeline, and it is worth asking here in particular because the four stages differ by two orders of magnitude in code size, and the two smallest of them turn out to carry the two largest effects we can measure.

Section 2 states the pipeline precisely. Section 3 records the implementation choices that determine the numbers. Section 4 measures the stages. Where a measurement turned up something broken rather than merely surprising, we say so and give the repair.

2 The pipeline

Let $R \in \mathbb{R}^{T \times N}$ be weekly returns for N funds.

2.1 Selection

Minimum spanning tree. Spearman correlations C are mapped to distances $d_{ij} = \sqrt{2(1 - C_{ij})}$, a complete graph is built on those distances, and its minimum spanning tree (Kruskal, 1956) is computed. The surviving subset is the set of *leaves* — vertices of degree one:

$$S = \{\, i : \deg_{\mathrm{MST}}(i) = 1 \,\}. \tag{1}$$

The rationale is that a leaf is a fund the tree could not place at the centre of a cluster, so leaves are the periphery of the correlation structure. The operation is applied repeatedly — n passes, each on the survivors of the last — with n a user parameter.

Two diagnostics are returned with the subset: the average correlation among the survivors, and the Portfolio Diversification Index of Rudin and Morgan (2006),

$$\mathrm{PDI} = 2 \sum_{k=1}^{n} k\, w_k - 1, \qquad w_k = \frac{\lambda_k}{\sum_j \lambda_j}, \tag{2}$$

where $\lambda_1 \geq \cdots \geq \lambda_n$ are the eigenvalues of the subset's correlation matrix. PDI equals 1 for a perfectly correlated block and n for n independent assets, so it reads as an effective number of bets.

Clustering. The alternative maps the same correlations to distances, runs complete-linkage hierarchical clustering into k groups, and keeps the m funds with the highest Sharpe ratio in each group, giving at most km names. Unlike (1) this stage conditions on realised performance, which is what Section 4.2 is about.

2.2 Moments and scenarios

For rebalancing period p the moments are estimated on a rolling window that ends where period p begins: with $T_{\text{train}} = T - T_{\text{test}}$, the window is rows $[4p,\ T_{\text{train}} + 4p)$. The covariance is the biased sample covariance shrunk towards a scaled identity with the Ledoit and Wolf (2004) intensity, and the mean is the plain window mean.

Scenarios are 4-week compounded returns, 2nd-stage inputs to the optimiser, generated one of two ways:

$$\text{Monte Carlo:}\quad s_j^{(p)} = \prod_{u=1}^{4}\left(1 + z_{j,u}^{(p)}\right) - 1, \qquad z_{j,u}^{(p)} \sim \mathcal{N}(\mu_p, \Sigma_p), \tag{3}$$

$$\text{Bootstrap:}\quad s_j^{(p)} = \prod_{u=1}^{4}\left(1 + R_{\tau_{j,u},\cdot}\right) - 1, \qquad \tau_{j,u} \sim \text{Unif}\{4p, \ldots, T_{\text{train}} + 4p\}. \tag{4}$$

The bootstrap draws whole *rows*, so the cross-sectional dependence of a given week is preserved exactly; the four weeks within a scenario are drawn independently, so serial dependence is not (Politis and Romano, 1994).

2.3 Target and optimisation

The risk limit for period p is the CVaR_α, at $\alpha = 5\%$, of an equal-weight portfolio of the benchmark's assets under *bootstrap* scenarios of the benchmark — irrespective of which generator supplies the optimiser's own scenarios.

Given scenarios $s^{(p)} \in \mathbb{R}^{J \times n}$ with equal probabilities, previous holdings x_{old}, cash c, and the period's target θ_p, the programme solved is

$$\begin{aligned} \max_{x \geq 0} \quad & \bar{\mu}_p^\top x \\ \text{s.t.} \quad & \zeta + \frac{1}{J\alpha}\sum_j \left[-s_j^{(p)\top} x - \zeta\right]_+ \leq \theta_p \cdot V_p, \\ & \mathbf{1}^\top x + \kappa \|x - x_{\text{old}}\|_1 = \mathbf{1}^\top x_{\text{old}} + c, \\ & x \leq w_{\max}\, \mathbf{1}^\top x, \end{aligned} \tag{5}$$

with ζ the free Value-at-Risk variable, κ the linear transaction cost rate and V_p the portfolio value entering the period. This is the Rockafellar and Uryasev (2000) linearisation, expressed in CVXPY (Diamond and Boyd, 2016) with the hinge realised as an auxiliary non-negative vector. The Markowitz alternative replaces the CVaR constraint by $x^\top \Sigma_p x \leq \theta_p$ with θ_p from a variance target.

Note the objective. $\bar{\mu}_p$ is the scenario mean, which is the rolling window mean — so the programme maximises extrapolated historical return subject to a tail constraint. It is the configuration Best and Grauer (1991) and Schmelzer and Hauser (2013) identify as the fragile one, and it is worth keeping in mind when reading Section 4.5.

3 Implementation

Four weeks is hard-wired. `n_iter = 4` appears independently in the moment generator, both scenario generators and the optimiser loop. The rebalancing frequency is not a parameter.

Two of the four stages are deterministic; two are not. Both selectors are deterministic: the MST is unique here, and the clustering is complete-linkage on a fixed distance matrix with a Sharpe-ranked pick. Everything downstream draws from an *unseeded* generator: `backtest`

Table 1: Repeated MST leaf selection on the selection window. *As returned* is the average correlation the function reports, which includes the unit diagonal; *off-diagonal* is the average over distinct pairs. The two PDI columns are (2) as implemented and as defined.

Pass	Kept	$\bar{\rho}$ as returned	$\bar{\rho}$ off-diagonal	PDI as coded	PDI from (2)
0	754	—	0.4375	—	—
1	313	0.4067	0.4048	2.24	20.43
2	147	0.3752	0.3709	2.66	20.91
3	77	0.3255	0.3166	3.65	20.05
4	41	0.2794	0.2614	5.53	16.75
5	19	0.2658	0.2250	7.92	11.23
6	11	0.2566	0.1823	6.53	7.87

constructs `ScenarioGenerator(np.random.default_rng())` with no seed. The lifecycle entry point does accept an `rng_seed`, but treats the value 0 — its own default, and the first value most users would pass — as "no seed".

Costs and caps are fixed inside the call. `backtest` passes $\kappa = 0.001$, $w_{\max} = 1$, $\alpha = 0.05$ and a budget of 100 as literals. They are parameters of the models it calls but not of the pipeline the user drives.

A solver failure is not reported as one. When (5) does not solve to optimality, the rebalancing routine pickles its inputs to `rebalance_inputs.pkl` in the working directory, logs, and returns `None` — which the caller immediately unpacks into four names. The user therefore sees a `TypeError` about unpacking `None` rather than the diagnostic that was written for them. In our runs the CLARABEL path never triggered this, so the observation is from reading, not from measurement.

The reported Sharpe ratio is a ratio of rounded numbers. `final_stats` rounds the annualised return and volatility to two decimals *before* dividing. For an annual return of 8.4% and volatility of 12.6% the exact ratio is 0.667 and the reported one is 0.615. All Sharpe ratios in this paper are computed from the portfolio value path instead, and we note the discrepancy because it is of the same order as some of the differences the pipeline is used to compare.

4 Results

The universe is 754 Danish funds and ETFs with weekly returns. All selection uses 2017-01-04 to 2020-12-30 (209 weeks); all evaluation uses 2021-01-06 to 2024-07-24 (186 weeks, 47 rebalancing periods). The benchmark is `iShares MSCI World ETF USD Dist`, which returned 13.26% a year at 14.32% volatility over the test window. Where a number depends on the unseeded generator we report the mean and range over eight identical runs.

4.1 Selection: what the MST passes do

Table 1 shows the selector doing what it claims. Each pass keeps roughly half of its input — 754 → 313 → 147 → 77 → 41 → 19 → 11 — and the average pairwise correlation falls monotonically from 0.44 to 0.18. As a mechanism for finding the periphery of a correlation graph it works.

The diagnostics reported alongside it are another matter, and they disagree about when to stop.

Table 2: PDI on reference matrices. For n independent assets the index should be n; for a perfectly correlated block, 1.

	$C = I_n$ (answer n)		$C = \mathbf{1}\mathbf{1}^\top$ (answer 1)	
n	As coded	From (2)	As coded	From (2)
5	4.000	5.000	NaN	1.000
20	19.000	20.000	NaN	1.000
50	49.000	50.000	NaN	1.000

Table 3: Equal-weight performance over the fixed test window (2021-01-06 to 2024-07-24) of subsets chosen by two selection windows. The second block lets the selection window overlap the evaluation period, as it does whenever a caller passes the full history.

Selection window	Selector	n	Return	Vol.	Sharpe
ends 2020-12-30	MST, 4 passes	41	+4.67%	9.22%	+0.51
ends 2020-12-30	clustering 5×5	25	+1.39%	8.23%	+0.17
ends 2024-07-24	MST, 4 passes	32	+3.52%	10.12%	+0.35
ends 2024-07-24	clustering 5×5	25	+8.12%	7.97%	+1.02

The average correlation is inflated, increasingly so. The value returned is the mean of the full correlation matrix, diagonal included, which overstates the average pairwise correlation by $(1 - \bar{\rho})/n$. At pass 1 that is 0.002 and irrelevant; at pass 6, with $n = 11$, it is 0.074 — the function reports 0.2566 where the average pairwise correlation is 0.1823. The bias grows exactly as the subset shrinks, which is to say exactly where the number is being used to argue that shrinking helped.

The PDI is computed from the wrong spectrum. Equation (2) needs the eigenvalues of the correlation matrix. The implementation instead fits a PCA (Pedregosa et al., 2011) to the correlation matrix *as if it were a data matrix* — rows treated as observations — so it eigendecomposes the covariance of C's rows rather than C. The centring that PCA performs costs one degree of freedom, and the sum in the code runs to $n - 1$ rather than n, dropping the last term. Both symptoms are visible on matrices whose answer is known:

Table 2 isolates the two faults. The identity case returns exactly $n - 1$, the missing term. The rank-one case returns NaN, because a matrix of identical rows has zero total variance after centring and the explained-variance ratio divides by it — accompanied by a runtime warning, and propagating into whatever the caller does next.

On real data the consequence is not a small bias but reversed advice. In Table 1 the coded PDI rises through pass 5 and would tell a user to keep iterating; the PDI of (2) peaks at pass 2 (20.91) and falls thereafter, saying the diversification of the subset is being destroyed from pass 3 onwards even as the average correlation keeps improving. The second reading is the coherent one: PDI is bounded above by the number of assets, and a subset of 11 funds cannot carry 20 independent bets however uncorrelated it is. Repairing (2) is four lines, and it changes the recommended number of MST passes from "as many as you like" to two.

4.2 Selection: the window is the whole game

Table 3 is the most important table in this paper and it is about an API, not an algorithm. The two selectors are called the same way — `mst(start_date, end_date, n_mst_runs)` and `clustering(start_date, end_date, n_clusters, n_assets)` — and neither signature nor

Table 4: Equal-weight 4-week return of the MST subset (41 funds), pooled over all 47 periods and 2000 scenarios. *Historical* is non-overlapping 4-week compounding of the realised weekly returns. The Gaussian reference for $\mathrm{CVaR}_{5\%}/\sigma$ is 2.063.

Source	Mean	SD	Skew	Excess kurt.	$\mathrm{CVaR}_{5\%}$
Monte Carlo (3)	+0.00499	0.03131	+0.098	+0.097	0.05831
Bootstrap (4)	+0.00489	0.03290	−1.051	+3.172	0.08664
Historical	+0.00493	0.03702	−2.839	+19.087	0.09137

docstring says that the window must end before the period you intend to evaluate.

For the MST selector it does not much matter: it uses only the correlation matrix, and extending its window into the test period moves the Sharpe ratio from +0.51 to +0.35, in the wrong direction. For the clustering selector it matters enormously: extending the window lifts the equal-weight Sharpe ratio from +0.17 to +1.02. The reason is `pick_cluster`, which keeps the highest-Sharpe funds in each cluster — so any overlap between the selection window and the evaluation window is a direct look-ahead on the quantity being evaluated.

The gap is 0.85 Sharpe, and it is larger than every difference between models and scenario generators reported below. A user comparing MST against clustering with the natural call — pass the whole history to both, then backtest — would conclude that clustering wins by a wide margin. On this data, with the windows kept honest, it loses by a wide margin under equal weight.

4.3 Scenarios: which tail

Table 4 answers a question the `scenarios_type` flag makes look cosmetic. The Monte Carlo branch samples a Gaussian and compounds four draws; four is not enough compounding to generate a tail, and the result is a distribution with skewness +0.10 and excess kurtosis +0.10. A CVaR constraint imposed on it is a variance constraint wearing different notation. The bootstrap branch, drawing historical weeks, recovers skewness −1.05 and excess kurtosis +3.17 — about 86% of the historical excess tail, measured as $(\mathrm{CVaR}_{\mathrm{boot}} - \mathrm{CVaR}_{\mathrm{MC}})/(\mathrm{CVaR}_{\mathrm{hist}} - \mathrm{CVaR}_{\mathrm{MC}})$. It falls short of history because it compounds four independently drawn weeks and so destroys the serial dependence that produces the −2.84 skewness of realised 4-week returns; a block bootstrap (Politis and Romano, 1994) would close part of that gap.

For the same portfolio, then, the Monte Carlo branch reports a 5% CVaR of 0.0583 where the bootstrap reports 0.0866: it understates the tail by 33%.

The mismatch does not cancel. It would be a smaller problem if the target and the constraint were computed the same way, because a uniformly understated risk measure on both sides would partly cancel. They are not: the target generator always bootstraps, whatever `scenarios_type` says. So selecting Monte Carlo scenarios leaves a fat-tailed target governing a thin-tailed constraint, and the effective risk budget is inflated by the ratio of the two — $1/0.667 = 1.50$ for this subset. The configuration that a user would pick to get a faster, smoother scenario set is the one that quietly loosens the risk limit by half again.

4.4 The target: the benchmark is the risk budget

Table 5 explains a result that is otherwise puzzling: why the optimised portfolios of Section 4.5 run at two to three times the volatility of equal weight on the same names. The risk limit is not a property of the selected subset. It is the tail of an equal-weight portfolio of the *benchmark*, and a single world-equity ETF has a much fatter 4-week tail (0.1103) than a basket of 41 diversified funds (0.0856) or 25 (0.0553). Choosing that benchmark therefore hands the optimiser a budget

Table 5: Mean $\mathrm{CVaR}_{5\%}$ of the equal-weight portfolio of each asset set, averaged over the 47 rebalancing periods. The target handed to the optimiser is the benchmark's row.

Asset set	Bootstrap	Monte Carlo	MC / bootstrap
Benchmark: MSCI World ETF (1)	0.1103	0.0876	0.794
MST subset (41)	0.0856	0.0571	0.667
Clustering subset (25)	0.0553	0.0450	0.813

Table 6: Optimised portfolios over the test window, each run eight times with identical arguments. Ranges are over those eight runs; volatility and drawdown are means. Rows marked *exact* returned the same value all eight times — the Markowitz path never consumes the scenarios it generates. The passive rows are deterministic.

Subset	Model / scenarios	Return (range)	Vol.	Sharpe (range)	Max DD
Passive references					
—	MSCI World ETF	+13.26%	14.32%	+0.93	−17.59%
all 754	equal weight	+4.99%	8.67%	+0.58	−17.00%
MST 41	equal weight	+4.67%	9.22%	+0.51	−16.77%
Clust. 25	equal weight	+1.39%	8.23%	+0.17	−20.03%
Optimised					
MST 41	CVaR / bootstrap	−10.79% [−20.7, −5.0]	18.23%	−0.58 [−0.88, −0.30]	−40.69%
MST 41	CVaR / MC	−13.23% [−17.5, −11.4]	21.37%	−0.62 [−0.84, −0.52]	−47.58%
MST 41	Markowitz / MC	−11.42% (exact)	20.69%	−0.55 (exact)	−39.56%
Clust. 25	CVaR / bootstrap	−10.17% [−18.8, +2.2]	25.63%	−0.36 [−0.64, +0.12]	−57.14%
Clust. 25	CVaR / MC	−2.77% [−8.7, +1.6]	21.80%	−0.13 [−0.38, +0.07]	−41.22%
Clust. 25	Markowitz / MC	−0.28% (exact)	17.95%	−0.02 (exact)	−35.10%

1.29 times what the MST subset already runs, and 2.00 times for the clustering subset — and the optimiser, whose objective is expected return, spends all of it.

This is design rather than defect: the model is a benchmark-relative one, and targeting the benchmark's CVaR is a coherent thing to want. It is worth stating plainly because the `benchmarks` argument is documented as a list of names to compare against, its only in-code comment is "Find Benchmarks' ISIN codes", and nothing signals that it is the single most consequential number in the call.

4.5 The backtest

Table 6 is the pipeline end to end, and its headline is uniform: every one of the six optimised configurations loses money over the test window, and every one trails equal weight on its own subset by a wide margin. Four things in it are worth separating from that.

The optimisation stage is where the value goes, and the mechanism is identified. Equal weight on the MST subset earns +4.67%; optimising over the same 41 names earns −10.8% to −13.2%. The gap is 1.0 to 1.5 Sharpe, which is larger than the difference between the two selectors, larger than the difference between CVaR and Markowitz, and larger than the difference between the two scenario generators. We would not read this as evidence about the models: the objective in (5) maximises the training-window mean, extrapolating 2017–2020 into a period in which the leadership of this universe changed completely, and one window cannot separate a bad estimator from a bad four years. What it does establish is that the stage carrying almost all of the code is also the stage carrying almost all of the risk of being wrong — which is the point DeMiguel et al. (2009) make about optimisation in general and Best and

Grauer (1991) make about this objective in particular.

The volatility is two to three times equal weight, as Table 5 predicts. Equal weight on the MST subset runs at 9.22% and on the clustering subset at 8.23%; the optimised versions run at 18% to 26%. The CVaR constraint is not being violated — it is being *met*, against a target set by a benchmark whose own tail is 1.29 to 2.00 times the subset's. Nothing about the calls in Table 6 announces that as a risk decision, and it is the largest one in them.

The selector ranking does not survive the optimiser, and cannot be measured anyway. The clustering subset is the worse of the two under equal weight (+0.17 against +0.51) and the better of the two after optimisation (−0.36, −0.13, −0.02 against −0.58, −0.62, −0.55). We report the reversal and decline to interpret it, because the run-to-run spread makes it unmeasurable: the clustering CVaR/bootstrap configuration alone spans −0.64 to +0.12 across eight identical calls.

An identical call is not repeatable — on the CVaR path. Because the scenario generator is constructed unseeded, the four CVaR rows of Table 6 are eight different portfolios each, with Sharpe spreads of 0.58, 0.32, 0.76 and 0.45. Those are wider than every difference between the six optimised configurations, which span −0.62 to −0.02. Reporting a single run — which is what the API invites, since `backtest` returns one statistics frame — is reporting a draw as if it were a mean. Threading a `seed` through to `ScenarioGenerator` is a two-line change and would make every number in this section checkable.

The Markowitz rows, by contrast, reproduce exactly, and for an instructive reason: `backtest` generates the scenario tensor before branching on the model, and the Markowitz branch passes only the deterministic rolling moments to its solver. The scenarios are built and discarded. That is why those rows have no range, and why they run in a fifth of the time.

5 Limitations

One universe, one window, one benchmark. The test period (2021–2024) contains a bond selloff, an equity drawdown and a concentrated recovery, and any conclusion about which stage helps is conditional on that sequence. The structural statements — the clustering selector leaks through its window, the benchmark sets the budget, Monte Carlo has no tail — would survive a different window; the levels in Table 6, and the sign of the optimisation's contribution, would not.

The optimised rows use 1000 scenarios and eight repetitions. Eight is enough to bound the noise, not to estimate its distribution, and 1000 scenarios is at the low end for a 5% tail measure — the CVaR is then an average over 50 scenarios, and its own sampling error contributes to the spread we attribute to the unseeded generator.

We did not examine the lifecycle module, the glide-path generator or the data acquisition layer, which together are about a third of the package. We also did not vary the parameters that `backtest` fixes internally: the transaction cost, the concentration cap and the 5% CVaR level are all left at their hard-coded values, and the transaction cost in particular ($\kappa = 0.001$ on a four-weekly rebalance) is a modelling choice worth its own study given the turnover (5) generates.

Finally, the comparison in Section 4.2 attributes the clustering selector's advantage under a contaminated window to look-ahead. That is the mechanism, but the size of the effect is specific to this window: a period in which past winners kept winning would show a smaller gap, and one in which they reversed could show a negative one.

6 Conclusion

Measured stage by stage on this universe and window, the two largest effects in `ifunnel` are not choices between models.

The largest is the selection *window*. Letting it overlap the evaluation period is worth +0.85 Sharpe to the clustering selector, because that selector ranks by realised Sharpe ratio; the MST selector, which reads only correlations, shows no such advantage. A user who passes the full history to `clustering(start_date, end_date, ...)` — the natural call, and the one nothing in the signature warns against — gets a look-ahead larger than any modelling decision in the package.

The second is the optimisation stage, which on this window costs 1.0 to 1.5 Sharpe against equal weight on the same names, in all six configurations. Two mechanisms are identifiable and both are fixable in principle: the objective extrapolates the training-window mean, and the risk budget comes from the benchmark rather than the subset — which is a coherent design for a benchmark-relative mandate and an invisible one in a call whose `benchmarks` argument reads as a list of things to plot against.

Three findings are repairs, and we would make them in this order. The CVaR backtest should take a seed: at present identical calls return Sharpe ratios spanning up to 0.76, which makes every comparison the pipeline exists to support unverifiable. The Portfolio Diversification Index should be computed from the eigenvalues of the correlation matrix and summed to n: as written it returns $n - 1$ for independent assets, NaN for a correlated block, and on this data recommends five MST passes where the correct index recommends two. And the CVaR target should be generated the same way as the constraint it governs; bootstrapping the target while the optimiser sees Gaussian scenarios inflates the effective risk budget by half.

None of this argues against the design. Reducing 754 funds to 41 by graph structure and then optimising a coherent tail measure over them is a reasonable thing to build, and the parts implementing published methods — the CVaR linearisation, the shrinkage, the MST reduction — do what they say. The lesson is about where the leverage sits. Two arguments naming a date range mattered more than every choice of model and scenario generator in the package.

A Reproducibility

All numbers come from the repository's own functions on `all_etfs_rets.parquet.gzip` under `src/ifunnel/financial_data/`. The selection stage:

```
from ifunnel import initialize_bot
from ifunnel.models.mst import minimum_spanning_tree
from ifunnel.models.clustering import cluster, pick_cluster

bot = initialize_bot()
sel = bot.weeklyReturns["2017-01-01":"2020-12-31"]
df = sel.copy()
for _ in range(4):
    mst, df, corr_avg, pdi = minimum_spanning_tree(df)

labels = cluster(sel, 5)
stat = bot.get_stat("2017-01-01", str(sel.index.date[-1]))
clu, _ = pick_cluster(data=sel, stat=stat, ml=labels, n_assets=5)
```

Table 2 compares the coded index against

```
w = np.linalg.eigvalsh(C)[::-1]
```

```
w = w / w.sum()
pdi = 2 * sum((i + 1) * w[i] for i in range(len(w))) - 1
```

Tables 4 and 5 use `MomentGenerator.generate_sigma_mu_for_test_periods` followed by `ScenarioGenerator(np.random.default_rng(0))` and `portfolio_risk_target`, with the primal `cvar` routine from `cvar_targets`. Table 6 calls

```
bot.backtest(start_train_date="2017-01-01", start_test_date="2020-12-31",
             end_test_date="2024-07-24", subset_of_assets=subset,
             benchmarks=["iShares MSCI World ETF USD Dist"],
             scenarios_type=scen, n_simulations=1000, model=model,
             solver="CLARABEL", lower_bound=0)
```

eight times per configuration. Because `backtest` returns only the rounded statistics frame, the value path was captured by wrapping `ifunnel.models.main.final_stats`, and the annualised return, volatility, Sharpe ratio and drawdown recomputed from it without rounding. That wrapper is the only modification to the package anywhere in this paper; the seed of the scenario generator inside `backtest` cannot be set from outside, which is why those rows are ranges.

References

Philippe Artzner, Freddy Delbaen, Jean-Marc Eber, and David Heath. Coherent measures of risk. *Mathematical Finance*, 9(3):203–228, 1999.

Michael J. Best and Robert R. Grauer. On the sensitivity of mean-variance-efficient portfolios to changes in asset means: Some analytical and computational results. *The Review of Financial Studies*, 4(2):315–342, 1991.

Victor DeMiguel, Lorenzo Garlappi, and Raman Uppal. Optimal versus naive diversification: How inefficient is the $1/n$ portfolio strategy? *The Review of Financial Studies*, 22(5):1915–1953, 2009.

Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7 (1):1–26, 1979.

Philippe Jorion. Bayes-Stein estimation for portfolio analysis. *Journal of Financial and Quantitative Analysis*, 21(3):279–292, 1986.

Pavlo Krokhmal, Jonas Palmquist, and Stanislav Uryasev. Portfolio optimization with conditional value-at-risk objective and constraints. *Journal of Risk*, 4(2):43–68, 2002.

Joseph B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.

Olivier Ledoit and Michael Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004.

Rosario N. Mantegna. Hierarchical structure in financial markets. *The European Physical Journal B*, 11(1):193–197, 1999.

Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.

Richard O. Michaud. The Markowitz optimization enigma: Is 'optimized' optimal? *Financial Analysts Journal*, 45(1):31–42, 1989.

J.-P. Onnela, A. Chakraborti, K. Kaski, J. Kertész, and A. Kanto. Dynamics of market correlations: Taxonomy and portfolio analysis. *Physical Review E*, 68:056110, 2003.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Dimitris N. Politis and Joseph P. Romano. The stationary bootstrap. *Journal of the American Statistical Association*, 89(428):1303–1313, 1994.

R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.

Alexander M. Rudin and Jonathan S. Morgan. A portfolio diversification index. *The Journal of Portfolio Management*, 32(2):81–89, 2006.

Thomas Schmelzer and Raphael Hauser. Seven sins in portfolio optimization. *arXiv preprint arXiv:1310.3396 [q-fin.PM]*, 2013.